Neuron Developer's Guide

A practical, end-to-end guide to building a **neuron** — a process that connects to the BigBrain gateway and executes capability-typed tasks on behalf of workflows.

This guide is the "how do I build one" companion to the reference material:

Doc	What it covers
This guide	The developer journey: concepts → first neuron → scope/idempotency decisions → testing → shipping
`NEURON_ARCHITECTURE.md`	System design & internals: gateway, routing, queues, leasing, policy hooks
Neuron SDK — TypeScript	Full TypeScript SDK API reference (npm README)
Neuron SDK — Python	Full Python SDK API reference (PyPI README)

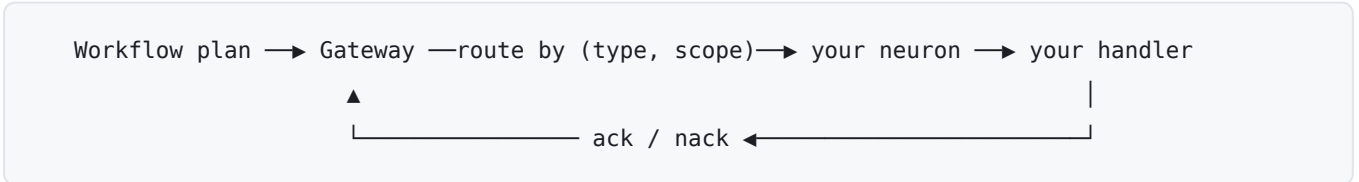
If you just want the API surface, go straight to the SDK README for your language. Read this guide first if you want the mental model and the decisions you'll have to make.

---

1. What a neuron is (in one paragraph)

A neuron is any process that can execute tasks. It advertises one or more **capabilities** (typed task handlers) to the gateway, holds a persistent connection, and receives work as **leases**. The built-in `apps/worker` is a neuron. The Holokai Desktop renderer is a neuron. A per-tenant agent running on a customer's box is a neuron. They all use the same SDK and the same wire protocol — the only thing that differs is *which* capabilities they advertise, *for whom* (scope), and *how* they connect (transport).

The gateway never runs your code. It routes a task to a neuron that has advertised a matching `(capabilityType, scope)`, hands it over as a lease, and waits for an `ack` or `nack`. Plan generation and orchestration stay centralized; execution fans out to neurons.



---

2. The five concepts you must hold in your head

You cannot write a correct neuron without internalizing these. Everything else is API detail.

Capability

A named contract you fulfill. Three fields:

- **`type`** — namespaced identifier: `{owner}/{package}[/{name}][.{variant}]`, e.g. `core/script.python`, `desktop/mcp/drive.read`.
- **`scope`** — `'any'` or `{ onBehalfOf: userId }` (see below).
- **`concurrency`** — max tasks of this type you'll run in parallel.

Capability *types are routing addresses, not schemas.* Input/output schemas travel with each plan, not with the registration. See §6.

Scope

Who a capability advertisement is valid for. This is the single most important decision you'll make per capability.

- **`'any'`** — you handle this task for any user. Shared queue. Use for stateless, non-user-specific compute (run a script, make an HTTP call, call an LLM with a server key).
- **`{ onBehalfOf: userId }`** — you handle this task *only* for that user. Per-user queue. Use whenever the work touches a user-specific resource — an MCP connection authenticated as that user, their Google Drive, their Slack.

> **Why this matters:** a desktop client authenticated to Google Drive as user `abc` must never process a Drive task for user `xyz`. The gateway enforces this: your JWT's user binding is the *only* authority for `onBehalfOf`. You cannot advertise a scope you aren't authorized for — registration is rejected.

Lease

How a task is delivered. The gateway writes a DB row with a `leaseId` and an expiry, then sends you a `lease` frame. **The task is not dequeued — it's leased.** The broker message stays unacknowledged until you `ack`. If you disconnect, the broker redelivers it. This is the root of the idempotency requirement (§4).

Capability authorization

Capability types are namespaced (`{owner}/{package}/{name}` — `core/http`, `desktop/mcp/drive.read`, `org/{orgId}/erp.read`): pick a prefix that makes your capability's ownership legible and collision-free.

What you may *advertise* is authorized at the **org level** via admission-on-advertisement: the first time your org sees a capability type, the gateway creates a ledger entry for it. Under the default org setting (`auto_approve`) it is approved and binds immediately — no pre-registration of capability types is ever needed. If your org admin has flipped the org to `require_approval`, new types are rejected with `CAPABILITY_PENDING_APPROVAL` until approved in Moku (re-register or re-send `update-capability` after approval); an admin-denied type rejects with `CAPABILITY_DENIED` and stays denied. There are no per-neuron capability subsets — any enrolled neuron of an org may serve any org-approved capability. (Full model: `NEURON_AUTH_SPEC.md`.)

A `skill/<name>/<id>` capability is the one exception to "a row per type": it is a composition of other capabilities, declared with `composes` on the registration (protocol 1.2.0; the gateway must be at that version

before an SDK sends it), and it is admitted exactly when every composed type is. It never gets a catalog row of its own and is never sent to Moku's governance queue; a skill that composes a denied type rejects with `CAPABILITY_DENIED`, one that composes a pending type with `CAPABILITY_PENDING_APPROVAL`, and one that declares nothing with `CAPABILITY_COMPOSITION_REQUIRED`. A skill may not compose another skill.

User scope is authorized separately: a capability advertised `{ onBehalfOf: userId }` requires your JWT's user binding to match — see §2 Scope above.

Authentication — getting a token

Your `auth` callback supplies the bearer token for every POST and SSE open. Two deployment shapes (`NEURON_AUTH_SPEC.md`):

- **Desktop-embedded** — run under the signed-in user: exchange the user's Moku JWT for an `aud=bigbrain` token (RFC 8693) and return it from `auth`.
- **Headless / enterprise** — enroll once via Moku's device flow, then mint short-lived tokens from the stored credential. No static long-lived JWT:

```
npx @holokai/neuron-sdk enroll --moku-url https://moku.example --name my-neuron
```

```
import { createClientCredentialsAuth, defaultCredentialPath } from '@holokai/neuron-sdk

const neuron = new Neuron({
  gatewayUrl,
  neuronId,
  auth: createClientCredentialsAuth(defaultCredentialPath()),
});
```

`createClientCredentialsAuth` caches tokens against `expires_in`, refreshes proactively, re-reads the credential file on `invalid_client` (operator secret rotation without restart), and throws a terminal `CredentialRevokedError` if the credential was revoked in Moku — re-enroll to recover. Node-only entry point; Python parity is a planned follow-up.

Scaling out a fleet (autoscaling)

Enrollment establishes one **durable machine identity** — it is a one-time, admin-approved step, *not* something each autoscaled replica should do on its own. Don't auto-enroll per instance: the device flow needs a human to approve each code, and self-enrolling would mint a fresh Moku client identity on every scale-up (identity churn you'd have to revoke and audit).

Instead, **provision one identity and share it as a secret:**

1. Enroll a single fleet identity once (an admin approves it in Moku):

```
npx @holokai/neuron-sdk enroll --moku-url https://moku.example --name my-fleet
```

2. Put the resulting credential file into your orchestrator's secret store — a Kubernetes `Secret`, an ECS task secret, Vault, a cloud secret manager — and mount it into every replica.
3. Every replica authenticates identically, pointing at the mounted path:

```
const neuron = new Neuron({
  gatewayUrl,
  neuronId: `my-fleet-${process.env.HOSTNAME}`,      // distinct per replica
  auth: createClientCredentialsAuth('/etc/holokai/neuron-credentials.json'),
});
```

The replicas **share the client identity but not the session**: give each a distinct `neuronId` (the pod/task name works) so the gateway tracks each connection separately. The credential is shared; the session is not.

Because `createClientCredentialsAuth` re-reads the credential on `invalid_client`, you can **rotate the shared secret in place** (update the Secret; let replicas pick it up) without a redeploy. Keep the fail-fast posture: a replica whose secret isn't mounted should crash so the scheduler surfaces it — never silently self-provision a new identity.

> **Per-instance identity without a shared secret** — having each replica prove itself via the platform's own workload identity (AWS IAM/STS, GCP metadata, Kubernetes projected ServiceAccount tokens / SPIFFE, OIDC), so Moku issues a credential against an admin trust policy instead of a distributed secret — is a planned follow-up. It removes the shared-secret blast radius and gives each replica an ephemeral, individually-revocable identity.

Neuron ID

A stable id for the process/device, reused across restarts. The gateway uses it to correlate reconnects. A server uses something like `worker-${HOSTNAME}-${pid}`; a desktop app persists a UUID to disk. If your `neuronId` changes every restart, the gateway accumulates stale session state.

---

3. Your first neuron (TypeScript)

The smallest useful neuron: one `any`-scoped capability.

```ts
import { Neuron } from '@holokai/neuron-sdk';

const neuron = new Neuron({
  gatewayUrl: process.env.GATEWAY_URL!,          // e.g. https://bigbrain.holokai.dev
  neuronId: `worker-${process.env.HOSTNAME}`,    // stable per process
  auth: () => process.env.GATEWAY_TOKEN!,         // dev shortcut — see §2 Authentication
                                                  // (headless prod: createClientCredentialsAu
});

neuron.handle(
  { type: 'core/http', scope: 'any', concurrency: 8 },
  async (input, ctx) => {
    const { url, method = 'GET', headers = {}, body } =
      input as { url: string; method?: string; headers?: Record<string, string>; body?: strin

    // Always thread ctx.signal so the call aborts on cancel/shutdown.
    const resp = await fetch(url, { method, headers, body, signal: ctx.signal });
    return { status: resp.status, body: await resp.text() };
  },
);

await neuron.start();                             // opens SSE, registers, heartbeats

process.on('SIGTERM', () => neuron.stop({ drain: true, timeoutMs: 30_000 }));
```

That's a complete neuron. `start()` opens the connection, posts your capabilities, and begins the heartbeat loop. From then on, leases arrive and your handler runs.

The same thing in Python

```python
import asyncio, logging
from holokai_neuron_sdk import Capability, HandlerContext, Neuron

async def http_handler(input, ctx: HandlerContext):
    # cooperate with cancellation via ctx.cancelled where you do I/O
    return {"status": 200, "body": "..."}

async def main():
    neuron = Neuron(
        gateway_url="https://bigbrain.holokai.dev",
        neuron_id="my-neuron-1",
        auth=lambda: "<bearer token>",
        logger=logging.getLogger("neuron"),
    )
    neuron.register_handler(
        Capability(type="core/http", scope="any", concurrency=8),
        http_handler,
    )
    await neuron.start()
    try:
        await asyncio.Event().wait()
    finally:
        await neuron.stop(drain=True, timeout=30)

asyncio.run(main())
```

Both SDKs speak the identical wire protocol (`packages/neuron-protocol`). Pick the language that fits where your capability lives.

---

4. The idempotency contract (read this twice)

> **Leases are at-least-once. A task that completed its side effect but failed to `ack` will run again.**

This is not a framework bug — it's the consequence of lease semantics. If your neuron crashes, the network drops, or the gateway restarts after your handler sent the email but before the ack landed, the broker redelivers the task and your handler runs a second time.

The framework does **not** checkpoint mid-task. Idempotency is *your* contract. Concretely:

- **Use capability-native idempotency keys.** Google Drive `requestId` , HTTP `Idempotency-Key` , Stripe idempotency keys, a dedup row keyed on `taskId` .
- **Make the safe operations safe to repeat.** Reads, pure transforms, and upserts are naturally idempotent. Sends, charges, and appends are not — guard them.
- **`ctx.task` carries a stable `taskId`** you can use as your dedup key when the downstream system has nothing better.

If you take one thing from this guide: **every handler that causes a side effect must be safe to run twice.**

---

5. Classifying failures correctly

How you signal failure determines whether the task retries, fails the workflow, or is treated as a benign cancel. Getting this wrong means either infinite retries of an unrecoverable task or a permanent failure of something transient.

You want…	TypeScript	Python	Wire kind	Effect
Success	`return output`	`return output`	`ack`	Output validated, workflow advances
Retry (transient)	`throw err`	raise any exception	`nack: retryable`	Broker redelivers
Permanent fail	`ctx.fail(reason, { code })`	`ctx.fail(reason, code=...)`	`nack: terminal`	Workflow fails with your reason
Cancelled	`throw` the `AbortError`	raise `CancelledError`	`nack: cancelled`	Benign, not a failure
Bad shape	*(automatic)*	*(automatic)*	`nack: schema-mismatch`	Structured "expected X got Y"

Rules of thumb:

- **5xx, network blip, rate limit → throw/raise.** It might work next time.
- **400, invalid credentials, malformed input you can't fix → `ctx.fail()`.** Retrying is pointless; fail fast with a `code` so the UI can explain it.
- **Cancellation → let it propagate.** When `ctx.signal` fires (TS) or `ctx.cancelled` is set (Python), stop promptly and let the `AbortError` / `CancelledError` bubble. The SDK turns it into a benign `cancelled` nack.

```
neuron.handle({ type: 'core/llm', scope: 'any', concurrency: 4 }, async (input, ctx) => {
  try {
    return await llmClient.complete({ ...input, signal: ctx.signal });
  } catch (err) {
    if ((err as Error).name === 'AbortError') throw err;          // cancellation
    if ((err as any).status === 400)
      ctx.fail('invalid LLM request', { code: 'INVALID_REQUEST' }); // terminal
    throw err;                                                      // retryable
  }
});
```

Retry policy itself lives on the workflow (server-side), never in your neuron. Your only job is to classify the outcome.

---

6. Schemas: where they live and who validates

There is **no global capability registry.** Schemas are owned per-plan:

- The client that initiates a workflow owns the **handler catalog** — the set of capability types available in its context, each with input/output schemas. The desktop knows its MCP tools; the server knows its bundled handlers.
- At plan creation the catalog is posted alongside the goal, the planner emits a typed plan, validates it against the catalog, and **snapshots the catalog with the plan.** Replay and debugging use the schemas valid at plan time.
- At execution, the **SDK** validates: input against `inputSchema` before invoking your handler, output against `outputSchema` before acking. A mismatch becomes a `schema-mismatch` nack automatically — your handler is never called on bad input.

In TypeScript, prefer `defineCapability()` with zod 4 so your handler types are inferred and your refinements run at runtime:

```
import { defineCapability } from '@holokai/neuron-sdk';
import { z } from 'zod';

const pythonCap = defineCapability({
  type: 'core/script.python',
  description: 'Run a Python script in a sandbox',
  input: z.object({ code: z.string(), args: z.record(z.string(), z.unknown()).optional() }),
  output: z.object({ stdout: z.string(), exitCode: z.number() }),
  scope: 'any',
  concurrency: 4,
});

neuron.handle(pythonCap, async (input, ctx) => {
  // input is typed: { code: string; args?: Record<string, unknown> }
  return { stdout: '...', exitCode: 0 };
});
```

The zod schema runs as the runtime validator (so `.refine()` works); a draft-07 JSON Schema projection is shipped to the gateway/planner. A schema with no JSON Schema form (`z.date()` , `.transform()` , `z.custom()`) makes `defineCapability` throw, naming the capability. The raw `{ inputSchema, outputSchema }` form is still accepted when you don't have zod on hand.

---

7. The HandlerContext toolkit

Every handler's second argument. Use all of it.

Member	Use it for
`ctx.task`	Full task metadata, incl. stable `taskId` for dedup keys
`ctx.leaseId`	Correlate logs/traces to a specific delivery
`ctx.signal` (TS) / `ctx.cancelled` (Py)	Abort I/O on cancel/shutdown — **thread it into every fetch/HTTP call**
`ctx.log`	Logger scoped to this lease (pino-shaped in TS)
`ctx.progress({ percent, message, data })`	Non-blocking UI updates for long tasks
`ctx.fail(reason, { code })`	Terminal failure (throws internally; unreachable after)

```
ctx.progress({ percent: 50, message: 'downloading' });
ctx.log.info({ fileId }, 'fetched metadata');
const resp = await fetch(url, { signal: ctx.signal });  // aborts cleanly on cancel
```

---

8. Choosing a transport

The `Neuron` class only ever talks to a `Transport`. Two ship in the box.

`HttpTransport` (default)

SSE inbound, HTTPS POST outbound, exponential-backoff reconnect (500ms → 30s), auth-callback token refresh with one retry on 401. This is what you get by passing `gatewayUrl` + `auth` to `Neuron`. Use it for any neuron talking to a *remote* gateway — servers, on-prem agents, standalone processes.

```
const neuron = new Neuron({ gatewayUrl, neuronId, auth });
```

`IpcTransport` (Electron in-process)

For a renderer-process neuron inside an Electron app that hosts BigBrain in the main process. Leases arrive over Electron IPC, not the network — **no JWT in the renderer**, no reconnect, no keep-alive. The main process attaches auth when it forwards POSTs to the embedded Fastify gateway via `app.inject()`.

```
import { Neuron } from '@holokai/neuron-sdk';
import { IpcTransport } from '@holokai/neuron-sdk/ipc';

const transport = new IpcTransport({ bridge: window.bigbrainNeuron, neuronId });
const neuron = new Neuron({ neuronId, transport });
```

The preload must expose a `NeuronIpcBridge` (`attach` / `detach` / `subscribe` / `invoke`) via `contextBridge` — see the SDK README for the exact wiring. The SDK itself has no Electron dependency.

> **Decision rule:** remote gateway → `HttpTransport`. Same Electron process as the gateway → `IpcTransport`. Anything else (WebSocket, native messaging) → implement the `Transport` interface; the rest of the SDK is unchanged.

---

9. Concurrency, heartbeats, and graceful shutdown

- **Concurrency** is per capability and maps to AMQP `prefetch_count`. The broker stops sending once you hold N unacked leases — backpressure is handled for you. Change it at runtime with `updateCapability({ type, concurrency })` (e.g. desktop on AC power vs. battery). In-flight tasks finish at the old limit; new slots open at the new one.
- **Heartbeats** post every ~10s with your in-flight lease list. They renew all active lease expiries in one round-trip and let the gateway detect a wedged neuron. **Only running leases appear in the heartbeat** — queued-but-waiting leases are excluded so the reaper doesn't extend their TTL. You don't manage this; the SDK does.
- **Shutdown** should be graceful: `neuron.stop({ drain: true, timeoutMs })` refuses new leases, lets in-flight handlers finish (up to the timeout, then aborts them via the signal), and nacks queued-but-unstarted leases as `retryable` so the broker redelivers immediately. Always wire it to `SIGTERM` / `SIGINT`.

---

10. Local development & testing

Run against a local gateway

The gateway is mounted on `apps/api`. Bring up the local stack (see `LOCAL_MODE.md` for `embedded` vs `remote` infra), point your neuron's `gatewayUrl` at it, and supply a dev JWT via your `auth` callback.

The neuron-tester

`apps/neuron-tester` is a reference neuron + harness — the fastest way to see a neuron register, take a lease, and ack against a running gateway. Read its README for the exact run commands; use it as a copyable template for a new neuron.

Unit-testing handlers

A handler is just `(input, ctx) => output`. Test it in isolation by passing a fake `ctx` — a stub `signal` / `cancelled`, a no-op `progress`, a capturing `log`. No gateway needed. For transport-level tests, the HTTP shape accepts a `fetch` override so you can drive frames synthetically.

Watch the wire

`Neuron` extends an `EventEmitter`. The `frame` event is a firehose of every parsed inbound SSE frame — invaluable for a debug UI or an integration test:

```
neuron.on('frame', (f) => debugLog.append(f));
neuron.once('connection:registered', ({ sessionId, capabilities }) =>
  console.log(`registered ${capabilities} caps as ${sessionId}`));
neuron.on('connection:reconnecting', ({ attempt, delayMs, reason }) =>
  statusBar.show(`reconnecting #${attempt} in ${delayMs}ms (${reason})`));
```

Subscribe *before* `start()` if you care about connection-sequence events.

---

11. Shipping checklist

Before you call a neuron production-ready, confirm:

- ☐ **`neuronId` is stable** across restarts (persisted, not random per boot).
- ☐ **Namespace matches identity.** Your JWT is authorized for the `type` prefix you advertise (`core/*` needs a service identity; `desktop/*` / `user/*` need the matching user binding).
- ☐ **Scope is correct.** Anything touching a user-specific resource is `{ onBehalfOf: userId }`, not `'any'`.
- ☐ **Every side-effecting handler is idempotent** (capability-native key or `taskId` dedup).
- ☐ **Failures are classified** — retryable vs. terminal vs. cancelled — and terminal failures carry a `code`.
- ☐ **`ctx.signal` / `ctx.cancelled` is threaded into every I/O call.**
- ☐ **Schemas are declared** (`inputSchema` / `outputSchema` or `defineCapability` zod) so bad shapes nack before your handler runs.
- ☐ **`auth` refreshes on demand** — the transport calls it on every request and once more on 401; your callback must return a valid token each call. `createClientCredentialsAuth` handles caching/refresh for you; a custom callback must do its own.
- ☐ **Graceful shutdown** wired to `SIGTERM` / `SIGINT` with a drain timeout.
- ☐ **Logging** follows `LOGGING_STANDARDS.md` (JSON pino, reserved field names, redaction).

---

12. Troubleshooting quick reference

Symptom	Likely cause
Registration rejected, **HTTP 403**	Capability pending approval or denied for the org (`CAPABILITY_PENDING_APPROVAL` / `CAPABILITY_DENIED`), or `onBehalfOf` doesn't match the user in the JWT
`handle() must be called before start()`	Register all capabilities before `start()`; use `updateCapability` for runtime changes
`start() requires at least one handler`	No `handle()` call was made
Tasks never arrive	Check the `registered` log for *effective* capabilities (policy/auth may have filtered one); check `neuronId` stability; check heartbeats
Same task delivered repeatedly	`ack` POST failed — usually the output failed `outputSchema`, or the gateway was restarting
Tasks stuck in `assigned`	Heartbeats aren't reaching the gateway (SSE dropped, heartbeat POSTs failing, or `heartbeatIntervalMs` set absurdly high)
401 on every request	`auth` callback isn't returning a valid/fresh token
`CredentialRevokedError`	The enrolled client was disabled/deleted in Moku (or the credential file is missing) — re-enroll with `neuron-sdk enroll`

For the deeper failure-mode model (disconnect mid-task, capability collision, policy change mid-workflow, gateway restart), see `NEURON_ARCHITECTURE.md` § Failure Modes.

Where to go next

- **Building a real capability?** Copy `apps/neuron-tester` and the worked examples in the SDK README on npm.
- **Need the internals** (queue topology, lease reaper, policy hooks)? `NEURON_ARCHITECTURE.md`.
- **Python neuron?** Neuron SDK — Python.
- **Auth model** (service accounts vs. user JWTs, per-task authority)? See the auth section of `NEURON_ARCHITECTURE.md`.